Групповой проект

Команда из **2–4 человек** делает работающий проект на Python и защищает его на **семинаре 14**. Промежуточных сдач и обязательных итераций нет: команда сама распределяет работу и при необходимости обращается за помощью.

С чего начать

Выберите тему из списка или предложите свою. Зарегистрируйте её у своего семинариста **до 7 ноября**: сообщите состав команды и в двух-трёх предложениях — что будет делать программа. Канал регистрации сообщит семинарист.

Сразу определите, что покажете на защите: **какие данные и как получает программа, что с ними делает и какой результат выдаёт**.

Выбирайте объём, который успеете закончить. Одного полезного сценария достаточно. **В пределах одной семинарской группы темы не повторяются**; команды из разных групп могут выбрать одну и ту же тему.

Темы на выбор

Для расчётных тем команда может подготовить небольшие таблицы. Пользователь выбирает готовые данные или вводит новые через интерфейс. Для бота нужны Telegram и интернет; игру и обработку фотографий можно показать локально.

1. **Куда уходят деньги?** Студент хочет понять, на что тратит стипендию. Он добавляет покупку через интерфейс; программа сравнивает расходы с месячными лимитами и показывает суммы по категориям и диаграмму расходов.

2. **Насколько подорожала привычная корзина?** Покупатель выбирает два месяца и свой набор продуктов. Программа считает стоимость корзины в каждом месяце и рисует диаграмму товаров, которые дали наибольший рост расходов.

3. **Какие товары приносят магазину деньги?** Владелец небольшой точки загружает готовую выгрузку продаж, закупочных цен и возвратов. Программа считает выручку и прибыль до общих расходов по каждому товару и показывает диаграмму наиболее и наименее выгодных позиций.

4. **Сколько нужно продать, чтобы не работать в убыток?** Владелец кофейни вводит цену напитка, затраты на порцию и постоянные расходы за месяц. Программа находит необходимое число продаж и рисует график прибыли при разных объёмах продаж.

5. **Хватит ли денег на ближайшие платежи?** Владелец небольшого бизнеса вводит текущий остаток и даты ожидаемых поступлений и платежей. Программа рисует график остатка по дням и отмечает первую дату, когда денег не хватит.

6. **Какой тариф связи выбрать для курьеров?** Менеджер службы доставки выбирает данные об использовании связи курьерами и несколько тарифов. Программа учитывает плату сверх

пакета, считает общую месячную стоимость для команды и показывает тарифы на диаграмме, выделяя самый дешёвый.

7. **Что пора заказать в магазин?** Управляющий видит текущие остатки, продажи за последние дни и срок поставки. Программа оценивает спрос за время ожидания и показывает товары, которые закончатся до приезда новой партии.

8. **У кого закупиться дешевле?** Кафе нужно купить несколько товаров; поставщики предлагают разные цены, стоимость доставки и доступное количество. Программа сравнивает допустимые варианты закупки и выбирает самый дешёвый для заданного списка товаров.

9. **В каких регионах цены росли быстрее?** Студенческая редакция выбирает два года из подготовленных данных. Программа строит диаграмму изменения цен по регионам и показывает, где данных не хватает.

10. **Телеграм-бот для учёта расходов.** Студент записывает покупку в чат сразу после оплаты: сумму и категорию. Бот сохраняет запись, показывает расходы за месяц и сообщает, сколько осталось до лимита.

11. **Игра про небольшой магазин.** Игрок решает, сколько товара закупить и по какой цене продавать. По заданным правилам игра рассчитывает спрос, выручку, расходы и остатки; после нескольких ходов показывает итоговую прибыль.

12. **Распознавание лиц на фотографиях мероприятия.** Организатор хочет разложить портреты участников по папкам. Он выбирает подписанные снимки добровольцев и новые портреты; программа показывает фотографии с именами знакомых людей и отдельно отмечает неуверенные совпадения.

Для бота, игры и распознавания лиц можно использовать готовые библиотеки. Содержательная часть проекта — работа с данными, правила и проверка результата.

Данные можно составить самим. Для темы про цены при желании можно взять опубликованные данные Росстата и подготовить из них небольшую таблицу.

Что должно работать

- Программа решает заявленную задачу от входных данных до результата.
- Человек взаимодействует с программой через страницу, окно, бота, игру или понятное меню в терминале: вводит данные или выбирает готовый набор и видит результат. Одного запуска скрипта или REST API без интерфейса недостаточно.
- Пользователю не нужно вызывать функции, вводить или читать JSON и словари Python, редактировать длинные таблицы или конфигурационные файлы. Готовый файл можно загрузить через интерфейс.
- Результаты подписаны и понятны без чтения кода. Графики и диаграммы рисуются как изображения и показываются в интерфейсе или открываются из него как PNG, а не выводятся псевдографикой в терминале.
- В коде есть содержательная логика на Python: обработка данных, расчёты, алгоритм или правила приложения. Каждый участник вносит значимый вклад.

- Основная логика отделена от интерфейса и внешних сервисов, чтобы её можно было проверить отдельно.
- У функций есть аннотации типов; неверный ввод и сбои внешних сервисов обрабатываются понятно.
- Основной сценарий и неверный ввод можно проверить через интерфейс на защите. Автоматические тесты — по желанию.
- Проект запускается в чистом окружении по инструкции из README.

Технологии выбирайте под задачу. Бот, веб-интерфейс, база данных, ИИ, Docker и облачный деплой нужны только тогда, когда помогают её решить. Количество библиотек и строк кода не добавляет баллов.

Что сдать

Сдайте исходный код и данные для демонстрации архивом или ссылкой на репозиторий. Если написали тесты, приложите и их. Git можно использовать для совместной работы, но он необязателен; историю коммитов не оцениваем. В коротком README укажите:

- какую задачу решает проект;
- как установить зависимости, запустить программу и воспользоваться ею;
- пример входных данных и ожидаемого результата;
- как устроены основные части программы и какие библиотеки используются;
- кто что сделал.

Пароли, токены и персональные данные в сдаваемые файлы не кладите. Для демонстрации подойдут открытые или вымышленные данные. Отдельные отчёты о ходе работы не нужны.

Если застряли, обращайтесь к преподавателю или ассистенту. При проблеме с кодом покажите небольшой пример, ожидаемый результат и текст ошибки.

Что показать на защите

1. Коротко объяснить задачу.
2. Через интерфейс ввести данные или выбрать готовые, запустить действие и показать понятный результат; если есть график или диаграмма — показать изображение.
3. Объяснить устройство кода и одно существенное техническое решение: например, выбор алгоритма, структуры данных или способа хранения.
4. Назвать ограничения и ответить на вопросы.

Каждый участник должен понимать задачу, основной сценарий, устройство проекта, ключевые решения и свой вклад.

Как оценивается проект

Проект даёт **40% оценки за семинары**; оценка за семинары имеет вес 0,6 в итоговой оценке курса. Важны:

- решение заявленной задачи и работающий запуск;
- качество кода и обработки ошибок;
- обоснованность технических решений;

- понимание проекта всеми участниками и ясная демонстрация.

Если хотя бы один участник не может объяснить, что сделала группа, оценка за проект — 0 для всей команды.

Полезные ссылки

- Git и структура Python-проекта — по желанию.
- Дополнительные упражнения.
- Программа курса и правила оценки.